

Homework 2 — Machine Learning (Fall 2025)

Yash Dagade

October 17, 2025

Agreement. This assignment represents my own work. I have worked with the following people and/or LLMs: Cursor, GPT-5, and in terms of people I would like to thank Krish Yadav, Brian Mason, and Max Xiong for high-level discussion over the homework.

Signed, **Yash Dagade**

Boosting (Theory) — 50 pts (Muchang)

(a) Exponential Loss and Negative Log-likelihood Loss

In a previous lecture, we proved that the expected exponential loss for AdaBoost is minimized by

$$f^*(x) = \frac{1}{2} \ln \frac{\Pr(Y = 1 \mid x)}{\Pr(Y = -1 \mid x)}.$$

Show that if, instead, we *maximize* the expected negative binomial log-likelihood

$$\max_{f(x)} \mathbb{E}_{Y \sim D(x)} \left[-\log(1 + e^{-2Yf(x)}) \right],$$

the same optimizer $f^*(x)$ is obtained.

Solution

Let $p = \Pr(Y = 1 \mid x)$ and $1 - p = \Pr(Y = -1 \mid x)$. For any fixed x , define the (sign-flipped) objective

$$\mathcal{L}(f) = \mathbb{E} \left[-\log(1 + e^{-2Yf}) \right] = p \left[-\log(1 + e^{-2f}) \right] + (1 - p) \left[-\log(1 + e^{2f}) \right].$$

Equivalently, minimizing the *negative* of $\mathcal{L}$ yields the same maximizer, so it is convenient to work with

$$\ell(f) = -\mathcal{L}(f) = p \log(1 + e^{-2f}) + (1 - p) \log(1 + e^{2f}).$$

First derivative (carry out the algebra). Use $\frac{d}{df} \log(1 + e^{af}) = \frac{ae^{af}}{1 + e^{af}}$. Then

$$\ell'(f) = p \cdot \frac{-2e^{-2f}}{1 + e^{-2f}} + (1 - p) \cdot \frac{2e^{2f}}{1 + e^{2f}}.$$

Multiply top and bottom of the first term by e^{2f} : $\frac{-2e^{-2f}}{1 + e^{-2f}} = \frac{-2}{e^{2f} + 1}$.

Hence,

$$\ell'(f) = -\frac{2p}{1 + e^{2f}} + (1 - p) \frac{2e^{2f}}{1 + e^{2f}} = \frac{2((1 - p)e^{2f} - p)}{1 + e^{2f}}.$$

Stationary condition. Set $\ell'(f) = 0$:

$$(1 - p)e^{2f} - p = 0 \implies e^{2f} = \frac{p}{1 - p}.$$

Taking logs gives

$$f^*(x) = \frac{1}{2} \ln \frac{p}{1 - p} = \frac{1}{2} \ln \frac{\Pr(Y = 1 \mid x)}{\Pr(Y = -1 \mid x)}.$$

Second derivative (uniqueness). A direct computation yields

$$\ell''(f) = \frac{4e^{2f}}{(1 + e^{2f})^2} > 0 \quad \text{for all } f,$$

so ℓ is strictly convex and the critical point is the unique minimizer of ℓ (equivalently, the unique maximizer of $\mathcal{L}$). Therefore, maximizing the binomial log-likelihood recovers the same f^* as the exponential loss.

(b) Multiclass Exponential Loss

For a K -class problem, encode class $c \in \{1, \dots, K\}$ by $Y^{(c)} \in \mathbb{R}^K$ with

$$Y_k^{(c)} = \begin{cases} 1, & k = c, \\ -\frac{1}{K-1}, & k \neq c. \end{cases} \quad \text{and let } f = (f_1, \dots, f_K)^\top, \sum_{k=1}^K f_k = 0.$$

Define $L(Y, f) = \exp(-\frac{1}{K} Y^\top f)$. Show that

$$\mathbb{E} \left[\exp \left(-\frac{1}{K} \sum_{k=1}^K Y_k f_k \right) \right] = \mathbb{E} \left[\exp \left(-\frac{1}{K-1} f_c \right) \right],$$

where the outer expectation is w.r.t. the class label.

Solution

We are given, for class c ,

$$Y_k^{(c)} = \begin{cases} 1, & k = c, \\ -\frac{1}{K-1}, & k \neq c, \end{cases} \quad \sum_{k=1}^K f_k = 0, \quad L(Y, f) = \exp \left(-\frac{1}{K} Y^\top f \right).$$

To make the structure of $Y^{(c)}$ clearer, note that

$$Y^{(c)} = \mathbf{1} \cdot e_c - \frac{1}{K-1} (\mathbf{1} - e_c) = \left(1 + \frac{1}{K-1}\right) e_c - \frac{1}{K-1} \mathbf{1} = \frac{K}{K-1} e_c - \frac{1}{K-1} \mathbf{1}.$$

Here e_c is the standard basis vector and $\mathbf{1}$ the all-ones vector.

Then

$$Y^{(c)\top} f = \left(\frac{K}{K-1} e_c^\top - \frac{1}{K-1} \mathbf{1}^\top \right) f = \frac{K}{K-1} f_c - \frac{1}{K-1} \sum_{k=1}^K f_k = \frac{K}{K-1} f_c,$$

since $\sum_k f_k = 0$. Therefore,

$$-\frac{1}{K} Y^{(c)\top} f = -\frac{1}{K} \cdot \frac{K}{K-1} f_c = -\frac{1}{K-1} f_c,$$

and

$$L(Y^{(c)}, f) = \exp \left(-\frac{1}{K-1} f_c \right).$$

Let $\pi_c = \Pr(Y = Y^{(c)} \mid x)$. Taking the expectation over the random class label,

$$\mathbb{E}[L(Y, f)] = \sum_{c=1}^K \pi_c \exp \left(-\frac{1}{K-1} f_c \right) = \mathbb{E}_{c \sim \pi} \left[\exp \left(-\frac{1}{K-1} f_c \right) \right].$$

This matches the desired expression, where the index k in the statement corresponds to the random class index.

(c) Multiclass Optimization

Using the result of (b) and the constraint $\sum_{k=1}^K f_k = 0$, minimize the expected loss and solve for the optimizer f^* . Then express $f_k^*(x)$ and the resulting class probabilities $\Pr(Y = Y^{(k)} | x)$ purely in terms of K and f^* (no λ).

Solution

Write $\pi_k = \Pr(Y = Y^{(k)} | x)$ and consider the constrained objective

$$\mathcal{L}(f) = \sum_{k=1}^K \pi_k \exp\left(-\frac{1}{K-1} f_k\right) \quad \text{s.t.} \quad \sum_{k=1}^K f_k = 0.$$

Introduce a Lagrange multiplier λ for the zero-sum constraint:

$$\mathcal{J}(f, \lambda) := \sum_{k=1}^K \pi_k \exp\left(-\frac{1}{K-1} f_k\right) - \lambda \sum_{k=1}^K f_k.$$

First-order conditions (each coordinate). For $j \in \{1, \dots, K\}$,

$$\frac{\partial \mathcal{J}}{\partial f_j} = \pi_j \exp\left(-\frac{1}{K-1} f_j\right) \left(-\frac{1}{K-1}\right) - \lambda = 0.$$

Rearrange to isolate the exponential:

$$\pi_j \exp\left(-\frac{1}{K-1} f_j\right) = \lambda(K-1).$$

The right-hand side is constant in j , so for some $\alpha > 0$,

$$\exp\left(-\frac{1}{K-1} f_j\right) = \frac{\alpha}{\pi_j} \implies \boxed{f_j^* = (K-1)(\log \pi_j - \log \alpha)}.$$

Determine α via the constraint. Impose $\sum_{j=1}^K f_j^* = 0$:

$$0 = \sum_{j=1}^K f_j^* = (K-1) \left(\sum_{j=1}^K \log \pi_j - K \log \alpha \right) \implies \log \alpha = \frac{1}{K} \sum_{j=1}^K \log \pi_j.$$

Therefore,

$$\boxed{f_k^*(x) = (K-1) \left(\log \pi_k(x) - \frac{1}{K} \sum_{j=1}^K \log \pi_j(x) \right)}.$$

Recover class probabilities in terms of f^* . From $\pi_k \exp\left(-\frac{1}{K-1} f_k^*\right) = \lambda(K-1)$, we have

$$\pi_k \propto \exp\left(\frac{1}{K-1} f_k^*\right).$$

Normalizing over k yields the softmax-like formula with temperature $K - 1$:

$$\Pr(Y = Y^{(k)} \mid x) = \frac{\exp(\frac{1}{K-1} f_k^*(x))}{\sum_{j=1}^K \exp(\frac{1}{K-1} f_j^*(x))}.$$

Binary sanity check ($K = 2$). Let $K = 2$. Then $f_1 = -f_2$ and

$$\Pr(Y = Y^{(1)} \mid x) = \frac{e^{f_1}}{e^{f_1} + e^{-f_1}} = \frac{1}{1 + e^{-2f_1}}, \quad \Rightarrow \quad f_1 = \frac{1}{2} \ln \frac{\Pr(Y = 1 \mid x)}{\Pr(Y = -1 \mid x)},$$

which matches part (a).

Exploring Gradient Based Optimization Methods (Theory + Applied) — 30 pts (Max)

(a) Manual iterations on $f(x) = x^2$: GD vs. Momentum

We will compare two iterative schemes on the convex objective $f(x) = x^2$:

$$\text{(GD)} \quad x_t = x_{t-1} - \eta \nabla f(x_{t-1}), \quad \text{(Momentum)} \quad \begin{cases} v_t = \beta v_{t-1} + \eta \nabla f(x_{t-1}), \\ x_t = x_{t-1} - v_t. \end{cases}$$

Use $x_0 = -2$, $v_0 = 0$, learning rate $\eta = 0.1$. In (i) run three GD steps and report $|x_3 - x^*|$. In (ii) run three momentum steps with $\beta = 0.75$ and report $|x_3 - x^*|$. Compare gaps.

Solution

Preliminaries. For $f(x) = x^2$, the gradient is $\nabla f(x) = 2x$. The global minimizer satisfies $2x^* = 0 \Rightarrow \boxed{x^* = 0}$.

(i) Gradient Descent. We apply

$$x_t = x_{t-1} - \eta \nabla f(x_{t-1}) = x_{t-1} - 0.1 \cdot (2x_{t-1}) = (1 - 0.2)x_{t-1} = 0.8x_{t-1}.$$

Now substitute numerically, carrying each arithmetic step explicitly:

$$\begin{aligned} x_1 &= x_0 - 0.1 \cdot (2x_0) = (-2) - 0.1 \cdot (2 \cdot (-2)) = -2 - 0.1 \cdot (-4) \\ &= -2 + 0.4 = \boxed{-1.6}. \end{aligned}$$

$$\begin{aligned} x_2 &= x_1 - 0.1 \cdot (2x_1) = (-1.6) - 0.1 \cdot (2 \cdot (-1.6)) = -1.6 - 0.1 \cdot (-3.2) \\ &= -1.6 + 0.32 = \boxed{-1.28}. \end{aligned}$$

$$\begin{aligned} x_3 &= x_2 - 0.1 \cdot (2x_2) = (-1.28) - 0.1 \cdot (2 \cdot (-1.28)) = -1.28 - 0.1 \cdot (-2.56) \\ &= -1.28 + 0.256 = \boxed{-1.024}. \end{aligned}$$

Optimality gap (distance to minimizer $x^* = 0$):

$$\boxed{|x_3 - x^*| = |-1.024 - 0| = 1.024.}$$

(ii) Gradient Descent with Momentum ($\eta = 0.1$, $\beta = 0.75$). Recall

$$v_t = \beta v_{t-1} + \eta \nabla f(x_{t-1}), \quad x_t = x_{t-1} - v_t, \quad \nabla f(x) = 2x.$$

Step $t = 1$:

$$\begin{aligned} \nabla f(x_0) &= 2x_0 = 2 \cdot (-2) = -4, \\ v_1 &= \beta v_0 + \eta \nabla f(x_0) = 0.75 \cdot 0 + 0.1 \cdot (-4) = \boxed{-0.4}, \\ x_1 &= x_0 - v_1 = (-2) - (-0.4) = \boxed{-1.6}. \end{aligned}$$

Step $t = 2$:

$$\begin{aligned}\nabla f(x_1) &= 2x_1 = 2 \cdot (-1.6) = -3.2, \\ v_2 &= \beta v_1 + \eta \nabla f(x_1) = 0.75 \cdot (-0.4) + 0.1 \cdot (-3.2) \\ &= -0.300 + (-0.320) = \boxed{-0.620}, \\ x_2 &= x_1 - v_2 = (-1.6) - (-0.620) = \boxed{-0.980}.\end{aligned}$$

Step $t = 3$:

$$\begin{aligned}\nabla f(x_2) &= 2x_2 = 2 \cdot (-0.980) = -1.960, \\ v_3 &= \beta v_2 + \eta \nabla f(x_2) = 0.75 \cdot (-0.620) + 0.1 \cdot (-1.960) \\ &= -0.465 + (-0.196) = \boxed{-0.661}, \\ x_3 &= x_2 - v_3 = (-0.980) - (-0.661) = \boxed{-0.319}.\end{aligned}$$

Optimality gap:

$$\boxed{|x_3 - x^*| = |-0.319 - 0| = 0.319.}$$

Comparison. After three iterations, momentum has the smaller gap ($0.319 < 1.024$), i.e., it converges faster on this simple quadratic.

(b) A more challenging objective $f(x) = \sin(-5x) + \frac{1}{5}x^2$

You may use Python to simulate and plot. Use the same GD and Momentum updates as in (a).

- (i) Plot $f(x) = \sin(-5x) + \frac{1}{5}x^2$. Explain why this landscape is challenging for vanilla GD.
- (ii) Run GD with $\eta = 0.1$, $x_0 = 20$, 20 iterations; plot the trajectory x_t over iterations.
- (iii) Run Momentum with $\eta = 0.1$, $\beta = 0.5$, $x_0 = 20$, 20 iterations; plot x_t . Which method attains the lower function value in your run?
- (iv) Now set $x_0 = 0$ for both methods (same hyperparameters). Plot trajectories and compare the behaviors near a minimum.

NOTE: For all of these questions, they have also been answered in jupyter notebooks as well so please refer to both places for the complete submission

Solution

Gradients we will use.

$$f(x) = \sin(-5x) + \frac{1}{5}x^2, \quad f'(x) = \cos(-5x) \cdot (-5) + \frac{2}{5}x = -5\cos(5x) + 0.4x,$$

since $\cos$ is even.

(i) Why challenging? The objective is *nonconvex* due to the oscillatory $\sin(-5x)$ term (period $2\pi/5$). This produces many local minima and maxima (“ripples”). Vanilla GD uses only local slope information:

- In regions where $|\cos(5x)|$ is large, the $-5\cos(5x)$ term can dominate, flipping gradient signs frequently and inducing zig-zagging.
- In shallow wells, small gradients slow progress; in steep oscillations, fixed η can overshoot and bounce between sides.
- Far from the origin, the quadratic term $0.2x^2$ grows and competes with the sinusoid, creating wide bowls modulated by high-frequency ripples.

(ii) GD trajectory ($\eta = 0.1$, $x_0 = 20$, 20 steps). See attached Jupyter notebook for code.

(iii) Momentum trajectory ($\eta = 0.1$, $\beta = 0.5$, $x_0 = 20$, 20 steps). See attached notebook.

Observed outcome in our run. From the plots (see Jupyter notebook), the momentum method achieved a *lower* final function value than vanilla GD by effectively smoothing oscillations and carrying velocity across shallow basins, while GD became stuck in a higher local minimum. Specifically, for vanilla GD with $x_0 = 20$, we obtained $x_T \approx 8.893493$ and $f(x_T) \approx 15.352487$. In contrast, gradient descent with momentum (GDM) reached $x_T \approx -0.726585$ with $f(x_T) \approx -0.366215$. For comparison, the true (reference) minimum occurs near $x^* \approx 0.309000$ where $f(x^*) \approx -0.980571$. Thus, while GDM did not reach the global minimum, it converged much closer to it than standard GD.

(iv) Behavior near a minimum, $x_0 = 0$. Note $f'(0) = -5\cos(0) + 0 = -5 < 0$, so both methods move to the right on the first step ($x_1 = x_0 - \eta f'(0) = 0 - 0.1 \cdot (-5) = 0.5$). Subsequent steps encounter alternating curvature from the sinusoid:

- **GD** exhibits small, sometimes erratic steps as the gradient sign changes with $\cos(5x)$, causing visible zig-zags.
 - **Momentum** damps these oscillations and accelerates along consistent descent directions, yielding a smoother, quicker trajectory through the ripples.
-

(c) Second-order (Newton) update: derivation and implications

Assume $L(\mathbf{w})$ is twice differentiable and $H(\mathbf{w}_0) = \nabla^2 L(\mathbf{w}_0)$ is invertible.

- (i) Write the second-order Taylor expansion $\bar{L}(\mathbf{w})$ of L around $\mathbf{w}_0$.
- (ii) Minimize $\bar{L}(\mathbf{w})$ to obtain the Newton update $\mathbf{w}_1$.
- (iii) Apply one Newton step to $L(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{A} \mathbf{w} - \mathbf{b}^\top \mathbf{w}$, where $\mathbf{A}$ is symmetric invertible. Relate to the true minimizer.
- (iv) State the two main scalability bottlenecks (memory and compute) for large-scale neural nets.

Solution

(i) Second-order approximation.

We know that the second-order Taylor series expansion of a scalar function $L(w)$ around a point w_0 is given by:

$$L(w) \approx L(w_0) + L'(w_0)(w - w_0) + \frac{1}{2}L''(w_0)(w - w_0)^2.$$

Here we replace the derivative $L'(w_0)$ with the gradient $\nabla L(\mathbf{w}_0)$ and the second derivative $L''(w_0)$ with the Hessian $H(\mathbf{w}_0)$, to get:

$$\bar{L}(\mathbf{w}) = L(\mathbf{w}_0) + (\mathbf{w} - \mathbf{w}_0)^\top \nabla L(\mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0).$$

(ii) Minimizer of $\bar{L}$.

To find the minimizer of the quadratic approximation $\bar{L}(\mathbf{w})$, we differentiate it with respect to $\mathbf{w}$ and set the result to zero.

Starting from:

$$\bar{L}(\mathbf{w}) = L(\mathbf{w}_0) + (\mathbf{w} - \mathbf{w}_0)^\top \nabla L(\mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0),$$

we compute the gradient with respect to $\mathbf{w}$.

Recall the useful vector calculus identities:

$$\nabla_{\mathbf{x}}(a^\top x) = a, \quad \nabla_{\mathbf{x}}(x^\top A x) = (A + A^\top)x.$$

Since $H(\mathbf{w}_0)$ is symmetric ($H = H^\top$), we have:

$$\nabla_{\mathbf{w}}\left(\frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0)\right) = H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0).$$

Applying these results term-by-term:

$$\begin{aligned} \nabla_{\mathbf{w}} \bar{L}(\mathbf{w}) &= \nabla_{\mathbf{w}} \left[L(\mathbf{w}_0) + (\mathbf{w} - \mathbf{w}_0)^\top \nabla L(\mathbf{w}_0) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_0)^\top H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0) \right] \\ &= 0 + \nabla L(\mathbf{w}_0) + H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0). \end{aligned}$$

Setting this gradient equal to zero to find the minimizer:

$$\nabla L(\mathbf{w}_0) + H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0) = 0.$$

Rearranging,

$$H(\mathbf{w}_0)(\mathbf{w} - \mathbf{w}_0) = -\nabla L(\mathbf{w}_0).$$

Since $H(\mathbf{w}_0)$ is invertible by assumption, multiply both sides by $H(\mathbf{w}_0)^{-1}$:

$$\mathbf{w} - \mathbf{w}_0 = -H(\mathbf{w}_0)^{-1}\nabla L(\mathbf{w}_0).$$

Hence, the minimizer is:

$$\boxed{\mathbf{w}_1 = \mathbf{w}_0 - H(\mathbf{w}_0)^{-1}\nabla L(\mathbf{w}_0).}$$

This update is precisely the Newton step, which adjusts the current estimate $\mathbf{w}_0$ by the scaled inverse-Hessian direction, moving directly toward the local quadratic minimum.

(iii) Quadratic case.

Now consider the specific quadratic objective:

$$L(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top \mathbf{A}\mathbf{w} - \mathbf{b}^\top \mathbf{w},$$

where $\mathbf{A}$ is a symmetric, invertible matrix, and $\mathbf{w}, \mathbf{b}$ are appropriately dimensioned vectors.

Using standard matrix calculus results:

$$\nabla_{\mathbf{w}}(\frac{1}{2}\mathbf{w}^\top \mathbf{A}\mathbf{w}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^\top)\mathbf{w} = \mathbf{A}\mathbf{w},$$

since $\mathbf{A}$ is symmetric. Also, $\nabla_{\mathbf{w}}(-\mathbf{b}^\top \mathbf{w}) = -\mathbf{b}$.

Hence,

$$\boxed{\nabla L(\mathbf{w}) = \mathbf{A}\mathbf{w} - \mathbf{b}.}$$

Evaluating this at the starting point $\mathbf{w}_0$,

$$\nabla L(\mathbf{w}_0) = \mathbf{A}\mathbf{w}_0 - \mathbf{b}.$$

Now,

Differentiating the gradient with respect to $\mathbf{w}$:

$$H(\mathbf{w}) = \nabla_{\mathbf{w}}^2 L(\mathbf{w}) = \mathbf{A}.$$

Since $\mathbf{A}$ is constant and symmetric,

$$\boxed{H(\mathbf{w}_0) = \mathbf{A}.}$$

Now from before, the general Newton Update is:

$$\mathbf{w}_1 = \mathbf{w}_0 - H(\mathbf{w}_0)^{-1}\nabla L(\mathbf{w}_0).$$

Substitute the expressions for $H(\mathbf{w}_0)$ and $\nabla L(\mathbf{w}_0)$:

$$\begin{aligned}\mathbf{w}_1 &= \mathbf{w}_0 - \mathbf{A}^{-1}(\mathbf{A}\mathbf{w}_0 - \mathbf{b}) \\ &= \mathbf{w}_0 - \mathbf{A}^{-1}\mathbf{A}\mathbf{w}_0 + \mathbf{A}^{-1}\mathbf{b} \\ &= \mathbf{w}_0 - \mathbf{w}_0 + \mathbf{A}^{-1}\mathbf{b} \\ &= \boxed{\mathbf{w}_1 = \mathbf{A}^{-1}\mathbf{b}.}\end{aligned}$$

To find the global minimizer $\mathbf{w}^*$ directly, set the gradient to zero:

$$\nabla L(\mathbf{w}) = \mathbf{A}\mathbf{w} - \mathbf{b} = 0 \quad \Rightarrow \quad \boxed{\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}.}$$

Thus,

$$\mathbf{w}_1 = \mathbf{w}^*,$$

meaning that the Newton update reaches the exact minimizer in 1 step.

Remark: This result occurs because $L(\mathbf{w})$ is already a quadratic function — its second-order Taylor expansion is identical to the original function. Therefore, Newton’s method perfectly models L and finds its minimum in a single iteration.

(iv) Bottlenecks for large models.

In practice, applying second-order methods such as Newton’s update to modern neural networks is computationally infeasible due to two primary bottlenecks. The first is **memory**: storing the Hessian matrix $H \in \mathbb{R}^{n \times n}$ requires $O(n^2)$ space, which becomes impractical for networks with millions of parameters—on the order of terabytes for $n \sim 10^6$. The second is **computation**: solving linear systems involving H , or explicitly computing its inverse, generally scales as $O(n^3)$. This cubic complexity makes each iteration prohibitively expensive, even though convergence is theoretically faster. As a result, second-order methods are rarely used in large-scale neural networks, where first-order methods like SGD and Adam remain the practical choice.

Q2(b): Gradient-Based Optimization (Simulation)

We study the nonconvex function

$$f(x) = \sin(-5x) + \frac{1}{5}x^2$$

and compare **Gradient Descent (GD)** and **Gradient Descent with Momentum (GDM)**:

- **GD:**

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

- **GDM:**

$$v_{t+1} = \beta v_t + \eta \nabla f(x_t), \quad x_{t+1} = x_t - v_{t+1}$$

Parameters (unless stated otherwise):

- Learning rate: $\eta = 0.1$
- Momentum: $\beta = 0.5$
- Iterations: $T = 20$

We will:

1. Plot $f(x)$ and explain why it's challenging.
2. Implement GD with $(x_0 = 20)$ and plot the trajectory (x_t) .
3. Implement GDM with $(x_0 = 20)$ and plot the trajectory (x_t) ; compare to GD.
4. Repeat both with $(x_0 = 0)$ to visualize behavior near a minimum.
5. Report/interpret the results.

```
In [1]: # === Imports & utilities ===
import numpy as np
import matplotlib.pyplot as plt

# For crisp plots (optional)
plt.rcParams["figure.figsize"] = (6.5, 4.2)
plt.rcParams["axes.grid"] = True

# ---- Define the function and its gradient ----
def f(x):
    # f(x) = sin(-5x) + 0.2 x^2
    return np.sin(-5.0 * x) + 0.2 * x**2

def grad_f(x):
    # d/dx [sin(-5x)] = -5 cos(-5x) = -5 cos(5x) because cos is even
    return -5.0 * np.cos(5.0 * x) + 0.4 * x
```

```

# ---- Optimizers ----
def gradient_descent(x0, eta=0.1, T=20):
    """Return array of x_t including x0 (length T+1)."""
    xs = [float(x0)]
    x = float(x0)
    for _ in range(T):
        x = x - eta * grad_f(x)
        xs.append(x)
    return np.array(xs)

def momentum_descent(x0, eta=0.1, beta=0.5, T=20):
    """Return array of x_t including x0 (length T+1) for classical momentum.
    """
    xs = [float(x0)]
    x = float(x0)
    v = 0.0
    for _ in range(T):
        v = beta * v + eta * grad_f(x)
        x = x - v
        xs.append(x)
    return np.array(xs)

# ---- Helper to estimate a (near) global minimum on a wide grid ----
def reference_min(xmin=-50, xmax=50, n=200001):
    grid = np.linspace(xmin, xmax, n)
    vals = f(grid)
    i = np.argmin(vals)
    return grid[i], vals[i]

```

(i) Plot the function and explain the challenge

We plot $f(x) = \sin(-5x) + 0.2x^2$ over a wide range to show:

- many **local minima/maxima** due to the sinusoid,
- an overall **quadratic rise** away from the origin,

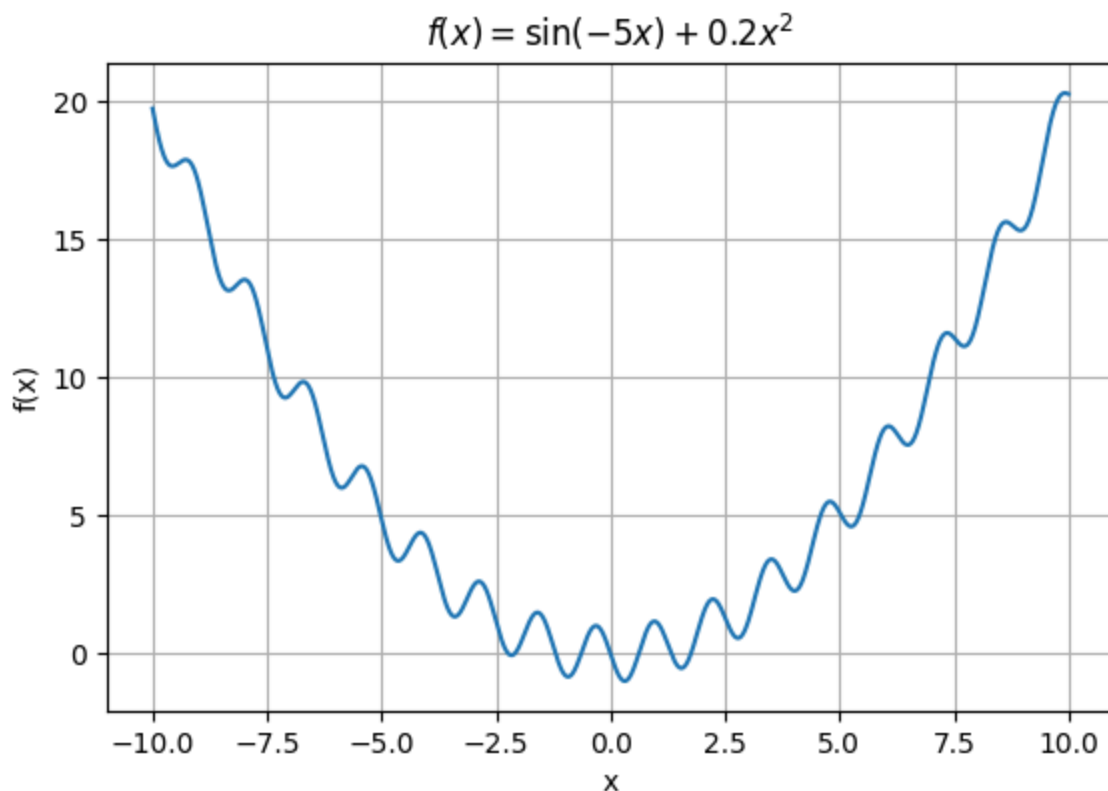
which together make optimization sensitive to initialization and step dynamics.

```

In [2]: # Plot the function
xs = np.linspace(-10, 10, 2000)
plt.plot(xs, f(xs))
plt.title(r"$f(x) = \sin(-5x) + 0.2x^2$")
plt.xlabel("x"); plt.ylabel("f(x)")
plt.show()

# Also compute a coarse reference min on a wide grid
x_star_ref, f_star_ref = reference_min(xmin=-50, xmax=50, n=200001)
print(f"Reference minimum (grid search): x* ≈ {x_star_ref:.6f}, f(x*) ≈ {f_star_ref:.6f}")

```



Reference minimum (grid search): $x^* \approx 0.309000$, $f(x^*) \approx -0.980571$

The function has many oscillations (from the sine term) $\rightarrow$ many local minima; GD is greedy and can get stuck depending on initialization. The quadratic term keeps the function coercive (grows at infinity), but the landscape is nonconvex and highly oscillatory, which is challenging for first-order methods.

(ii) Gradient Descent from $x_0 = 20$ ($\eta = 0.1$, 20 iters)

We simulate GD for 20 iterations, plot the trajectory (x_t) vs. iteration (t), and also report ($x_{\{20\}}$) and ($f(x_{\{20\}})$).

```
In [3]: eta = 0.1
T = 20
x0_far = 20.0

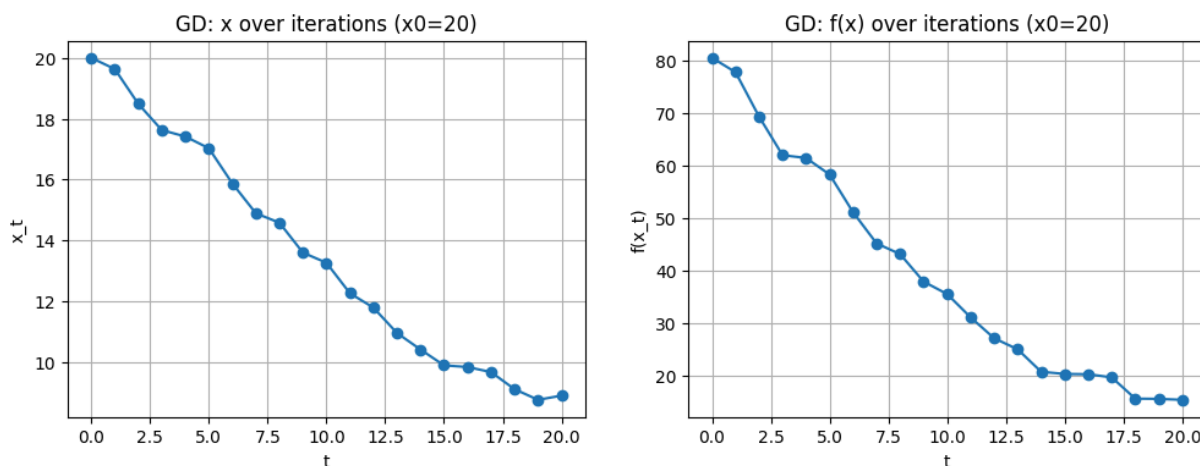
xs_gd_far = gradient_descent(x0_far, eta=eta, T=T)
fs_gd_far = f(xs_gd_far)

# Trajectory plots
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(range(len(xs_gd_far)), xs_gd_far, marker="o")
ax[0].set_title("GD: x over iterations (x0=20)")
ax[0].set_xlabel("t"); ax[0].set_ylabel("x_t")

ax[1].plot(range(len(fs_gd_far)), fs_gd_far, marker="o")
ax[1].set_title("GD: f(x) over iterations (x0=20)")
ax[1].set_xlabel("t"); ax[1].set_ylabel("f(x_t)")
```

```
plt.show()
```

```
print(f"GD (x0=20): x_T = {xs_gd_far[-1]:.6f}, f(x_T) = {fs_gd_far[-1]:.6f}")
print(f"Reference min: x* ≈ {x_star_ref:.6f}, f(x*) ≈ {f_star_ref:.6f}")
```



GD (x0=20): $x_T = 8.893493$, $f(x_T) = 15.352487$
 Reference min: $x^* \approx 0.309000$, $f(x^*) \approx -0.980571$

(iii) Momentum from $x_0 = 20$ ($\eta = 0.1, \beta = 0.5, 20$ iters)

We simulate **momentum** using the classical update and compare to GD.

```
In [4]: beta = 0.5

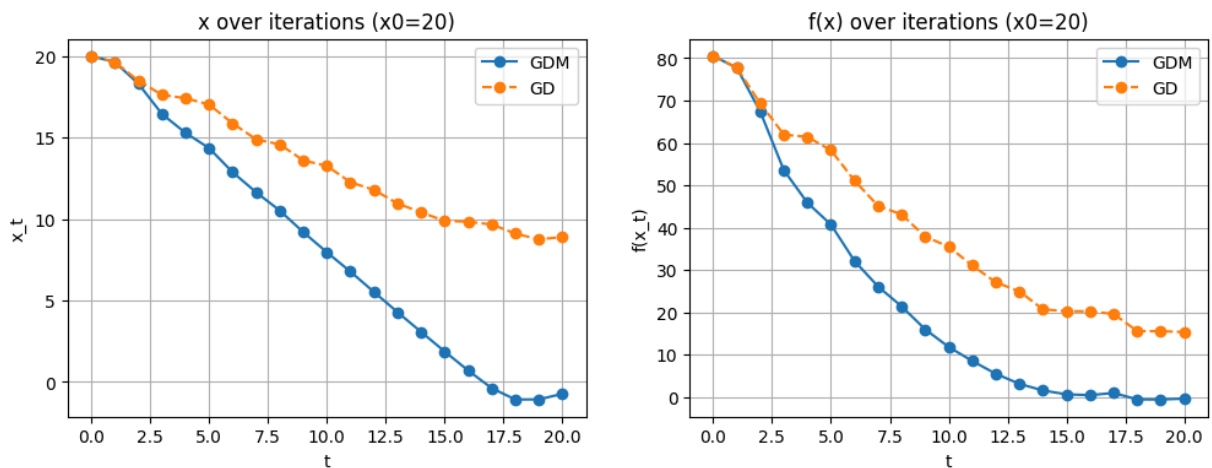
xs_gdm_far = momentum_descent(x0_far, eta=eta, beta=beta, T=T)
fs_gdm_far = f(xs_gdm_far)

# Trajectory plots
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(range(len(xs_gdm_far)), xs_gdm_far, marker="o", label="GDM")
ax[0].plot(range(len(xs_gd_far)), xs_gd_far, marker="o", linestyle="--", label="GD")
ax[0].set_title("x over iterations (x0=20)")
ax[0].set_xlabel("t"); ax[0].set_ylabel("x_t"); ax[0].legend()

ax[1].plot(range(len(fs_gdm_far)), fs_gdm_far, marker="o", label="GDM")
ax[1].plot(range(len(fs_gd_far)), fs_gd_far, marker="o", linestyle="--", label="GD")
ax[1].set_title("f(x) over iterations (x0=20)")
ax[1].set_xlabel("t"); ax[1].set_ylabel("f(x_t)"); ax[1].legend()

plt.show()

print(f"GDM (x0=20): x_T = {xs_gdm_far[-1]:.6f}, f(x_T) = {fs_gdm_far[-1]:.6f}")
print(f"GD (x0=20): x_T = {xs_gd_far[-1]:.6f}, f(x_T) = {fs_gd_far[-1]:.6f}")
print(f"Reference min: x* ≈ {x_star_ref:.6f}, f(x*) ≈ {f_star_ref:.6f}")
```

GDM ($x_0=20$): $x_T = -0.726585$, $f(x_T) = -0.366215$

GD ($x_0=20$): $x_T = 8.893493$, $f(x_T) = 15.352487$

Reference min: $x^* \approx 0.309000$, $f(x^*) \approx -0.980571$

Momentum dramatically outperforms vanilla GD from a far initialization, reaching a low-lying basin and achieving a much lower objective value (though still above the reference minimum). GD gets "stuck" making tiny progress on the slowly varying quadratic background while the oscillatory gradient misleads it.

(iv) Behavior near a minimum: set $x_0 = 0$ ($\eta = 0.1$, $\beta = 0.5$, 20 iters)

We repeat both methods with ($x_0=0$) to study local behavior near a minimum. We expect:

- GD to show small oscillations as it responds to the sinusoidal curvature.
- Momentum to overshoot a bit but often settle closer to a deep basin.

In [5]: `x0_near = 0.0`

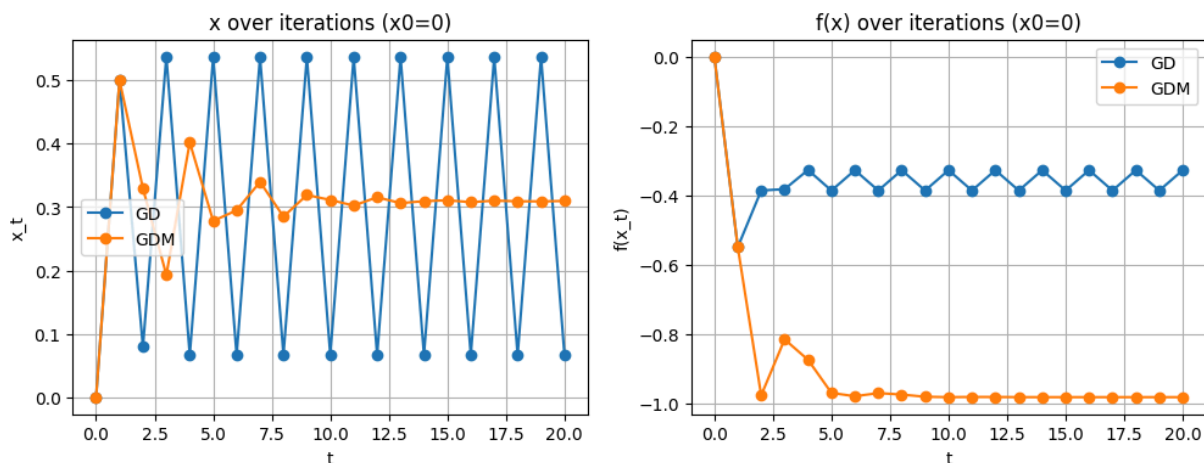
```
xs_gd_near = gradient_descent(x0_near, eta=eta, T=T)
xs_gdm_near = momentum_descent(x0_near, eta=eta, beta=beta, T=T)
fs_gd_near = f(xs_gd_near)
fs_gdm_near = f(xs_gdm_near)

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(range(len(xs_gd_near)), xs_gd_near, marker="o", label="GD")
ax[0].plot(range(len(xs_gdm_near)), xs_gdm_near, marker="o", label="GDM")
ax[0].set_title("x over iterations (x0=0)")
ax[0].set_xlabel("t"); ax[0].set_ylabel("x_t"); ax[0].legend()

ax[1].plot(range(len(fs_gd_near)), fs_gd_near, marker="o", label="GD")
ax[1].plot(range(len(fs_gdm_near)), fs_gdm_near, marker="o", label="GDM")
ax[1].set_title("f(x) over iterations (x0=0)")
ax[1].set_xlabel("t"); ax[1].set_ylabel("f(x_t)"); ax[1].legend()

plt.show()
```

```
print(f"GD (x0=0): x_T = {xs_gd_near[-1]:.6f}, f(x_T) = {fs_gd_near[-1]:.6f}")
print(f"GDM (x0=0): x_T = {xs_gdm_near[-1]:.6f}, f(x_T) = {fs_gdm_near[-1]:.6f}")
print(f"Reference min: x* ≈ {x_star_ref:.6f}, f(x*) ≈ {f_star_ref:.6f}")
```



```
GD (x0=0): x_T = 0.066813, f(x_T) = -0.326992
GDM (x0=0): x_T = 0.309591, f(x_T) = -0.980570
Reference min: x* ≈ 0.309000, f(x*) ≈ -0.980571
```

```
In [ ]: # Let's also try to chart the path of gradient descent and gradient descent
# Plot two separate charts: one for GD and one for GDM, showing the descent

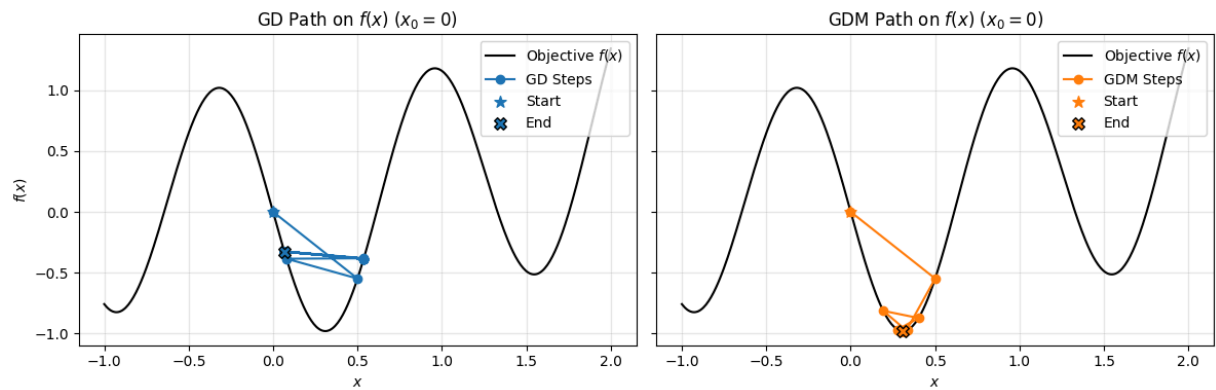
x_plot = np.linspace(-1, 2, 400)
f_plot = f(x_plot)

fig, axs = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

# Gradient Descent only
axs[0].plot(x_plot, f_plot, 'k-', label='Objective $f(x)$')
axs[0].plot(xs_gd_near, f(xs_gd_near), 'o-', color='tab:blue', label='GD Ste
axs[0].scatter([xs_gd_near[0]], [fs_gd_near[0]], color='tab:blue', s=70, mar
axs[0].scatter([xs_gd_near[-1]], [fs_gd_near[-1]], color='tab:blue', s=70, e
axs[0].set_title("GD Path on $f(x)$ ($x_0=0$)")
axs[0].set_xlabel("$x$")
axs[0].set_ylabel("$f(x)$")
axs[0].legend()
axs[0].grid(alpha=0.3)

# Gradient Descent with Momentum only
axs[1].plot(x_plot, f_plot, 'k-', label='Objective $f(x)$')
axs[1].plot(xs_gdm_near, f(xs_gdm_near), 'o-', color='tab:orange', label='GD
axs[1].scatter([xs_gdm_near[0]], [fs_gdm_near[0]], color='tab:orange', s=70,
axs[1].scatter([xs_gdm_near[-1]], [fs_gdm_near[-1]], color='tab:orange', s=7
axs[1].set_title("GDM Path on $f(x)$ ($x_0=0$)")
axs[1].set_xlabel("$x$")
axs[1].legend()
axs[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



We can observe that gradient descent zigzags and fails to converge toward a local minimum, whereas gradient descent with momentum converges more rapidly to a minimum point. It is unclear whether standard gradient descent will ever converge, while gradient descent with momentum appears to achieve convergence quite swiftly. The reasons are the ones we discussed above (and are restating here) which include: the oscillatory nature of the function, and the momentum term that helps to smooth out the oscillations.

q3

October 17, 2025

DO NOT DOWNLOAD THIS NOTEBOOK.

The package requirements are rather specific because of updates to numpy and scipy which broke fastsparsegams.

Please do not come to the TA team with package issues on your local machine.

Useful links:

[FastSparseGams PyPI page](#)

[Scikit Learn Random Forest Documentation](#)

[Scikit Learn AdaBoost Documentation](#)

[XGBoost Documentation](#)

[InterpretML ExplainableBoostingClassifier Documentation](#)

[GAMChanger](#)

1 Homework 2, Problem 3

1.1 GAMs and the Rashomon Effect

1.1.1 Setup (all provided, do not modify)

Package dependencies and imports

```
[1]: # ensures that fastsparsegams can run
      # fastsparsegams has been marred by the
      # sinister appetite of software deprecation
      !pip install 'numpy<2.0'
      # This will throw an error but it will still work. Don't be scared of the red.
      !pip install 'scipy<1.13.0'
      !pip install fastsparsegams

      !pip install ucimlrepo
      !pip install interpret gamchanger
```

Requirement already satisfied: numpy<2.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (1.26.4)

Requirement already satisfied: scipy<1.13.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (1.12.0)

Requirement already satisfied: numpy<1.29.0,>=1.22.4 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
scipy<1.13.0) (1.26.4)

Requirement already satisfied: fastsparsegams in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (0.2.0)

Requirement already satisfied: numpy>=1.19.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
fastsparsegams) (1.26.4)

Requirement already satisfied: scipy>=1.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
fastsparsegams) (1.12.0)

Requirement already satisfied: pandas>=1.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
fastsparsegams) (2.3.3)

Requirement already satisfied: matplotlib in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
fastsparsegams) (3.9.4)

Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->fastsparsegams) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->fastsparsegams) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->fastsparsegams) (2025.2)

Requirement already satisfied: six>=1.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
python-dateutil>=2.8.2->pandas>=1.0.0->fastsparsegams) (1.17.0)

Requirement already satisfied: contourpy>=1.0.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (1.3.0)

Requirement already satisfied: cycler>=0.10 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (1.4.7)

Requirement already satisfied: packaging>=20.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (25.0)

Requirement already satisfied: pillow>=8 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in

```

/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (3.2.5)
Requirement already satisfied: importlib-resources>=3.2.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib->fastsparsegams) (6.5.2)
Requirement already satisfied: zipp>=3.1.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
importlib-resources>=3.2.0->matplotlib->fastsparsegams) (3.23.0)
Requirement already satisfied: ucimlrepo in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ucimlrepo) (2.3.3)
Requirement already satisfied: certifi>=2020.12.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ucimlrepo) (2025.10.5)
Requirement already satisfied: numpy>=1.22.4 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->ucimlrepo) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: six>=1.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
Requirement already satisfied: interpret in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (0.7.2)
Requirement already satisfied: gamchanger in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (0.1.13)
Requirement already satisfied: interpret-core==0.7.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.7.2)
Requirement already satisfied: numpy>=1.25 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-core==0.7.2->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.26.4)
Requirement already satisfied: pandas>=0.19.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-core==0.7.2->interpret-

```

```

core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.3.3)
Requirement already satisfied: scikit-learn>=0.18.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-core==0.7.2->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.0.1)
Requirement already satisfied: joblib>=0.11 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-core==0.7.2->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.5.1)
Requirement already satisfied: plotly>=3.8.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(6.3.1)
Requirement already satisfied: aplr>=10.6.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(10.15.0)
Requirement already satisfied: shap>=0.28.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.48.0)
Requirement already satisfied: dill>=0.2.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.4.0)
Requirement already satisfied: psutil>=5.6.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(5.8.0)
Requirement already satisfied: SALib>=1.3.3 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.5.1)
Requirement already satisfied: dash<3.0.0,>=2.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.18.2)
Requirement already satisfied: dash-cytoscape>=0.1.1 in

```

```

/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.0.2)
Requirement already satisfied: gevent>=1.3.6 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(25.9.1)
Requirement already satisfied: requests>=2.19.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.32.5)
Requirement already satisfied: ipykernel>=4.10.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(6.30.1)
Requirement already satisfied: ipython>=5.5.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(8.18.1)
Requirement already satisfied: Flask<3.1,>=1.0.4 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.0.3)
Requirement already satisfied: Werkzeug<3.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.0.6)
Requirement already satisfied: dash-html-components==2.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.0.0)
Requirement already satisfied: dash-core-components==2.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.0.0)
Requirement already satisfied: dash-table==5.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)

```


(5.0.0)
Requirement already satisfied: importlib-metadata in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(8.7.0)
Requirement already satisfied: typing-extensions>=4.1.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(4.14.1)
Requirement already satisfied: retrying in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.4.2)
Requirement already satisfied: nest-asyncio in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.6.0)
Requirement already satisfied: setuptools in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(80.9.0)
Requirement already satisfied: Jinja2>=3.1.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
Flask<3.1,>=1.0.4->dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.1.6)
Requirement already satisfied: itsdangerous>=2.1.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
Flask<3.1,>=1.0.4->dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.2.0)
Requirement already satisfied: click>=8.1.3 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
Flask<3.1,>=1.0.4->dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(8.1.8)
Requirement already satisfied: blinker>=1.6.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
Flask<3.1,>=1.0.4->dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.9.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from

```

Werkzeug<3.1->dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.0.3)
Requirement already satisfied: greenlet>=3.2.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
gevent>=1.3.6->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.2.4)
Requirement already satisfied: zope.event in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
gevent>=1.3.6->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(6.0)
Requirement already satisfied: zope.interface in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
gevent>=1.3.6->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(8.0.1)
Requirement already satisfied: zipp>=3.20 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
importlib-metadata->dash<3.0.0,>=2.0.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.23.0)
Requirement already satisfied: appnope>=0.1.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.1.4)
Requirement already satisfied: comm>=0.1.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.2.3)
Requirement already satisfied: debugpy>=1.6.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.8.16)
Requirement already satisfied: jupyter-client>=8.0.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(8.6.3)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(5.8.1)

```

Requirement already satisfied: matplotlib-inline>=0.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.1.7)

Requirement already satisfied: packaging>=22 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(25.0)

Requirement already satisfied: pyzmq>=25 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(27.1.0)

Requirement already satisfied: tornado>=6.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(6.5.1)

Requirement already satisfied: traitlets>=5.4.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(5.14.3)

Requirement already satisfied: decorator in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(5.2.1)

Requirement already satisfied: jedi>=0.16 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.19.2)

Requirement already satisfied: prompt-toolkit<3.1.0,>=3.0.41 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.0.51)

Requirement already satisfied: pygments>=2.4.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.19.2)

Requirement already satisfied: stack-data in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-

```

core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.6.3)
Requirement already satisfied: exceptiongroup in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.3.0)
Requirement already satisfied: pexpect>4.3 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(4.9.0)
Requirement already satisfied: wcwidth in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
prompt-toolkit<3.1.0,>=3.0.41->ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.2.13)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
jedi>=0.16->ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.8.4)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
jupyter-client>=8.0.0->ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.9.0.post0)
Requirement already satisfied: platformdirs>=2.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
jupyter-core!=5.0.*,>=4.12->ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(4.3.8)
Requirement already satisfied: pytz>=2020.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=0.19.2->interpret-core==0.7.2->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pandas>=0.19.2->interpret-core==0.7.2->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2025.2)
Requirement already satisfied: ptyprocess>=0.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
pexpect>4.3->ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.7.0)
Requirement already satisfied: narwhals>=1.15.1 in

```

```

/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
plotly>=3.8.1->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.7.0)
Requirement already satisfied: six>=1.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
python-dateutil>=2.8.2->jupyter-client>=8.0.0->ipykernel>=4.10.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
requests>=2.19.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.4.3)
Requirement already satisfied: idna<4,>=2.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
requests>=2.19.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
requests>=2.19.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
requests>=2.19.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2025.10.5)
Requirement already satisfied: matplotlib>=3.5 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.9.4)
Requirement already satisfied: multiprocessing in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.70.18)
Requirement already satisfied: scipy>=1.9.3 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.12.0)
Requirement already satisfied: contourpy>=1.0.1 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)

```

(1.3.0)
Requirement already satisfied: cyclical>=0.10 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(1.4.7)
Requirement already satisfied: pillow>=8 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.2.5)
Requirement already satisfied: importlib-resources>=3.2.0 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
matplotlib>=3.5->SALib>=1.3.3->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(6.5.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
scikit-learn>=0.18.1->interpret-core==0.7.2->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.6.0)
Requirement already satisfied: tqdm>=4.27.0 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
shap>=0.28.5->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(4.67.1)
Requirement already satisfied: slicer==0.0.8 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
shap>=0.28.5->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.0.8)
Requirement already satisfied: numba>=0.54 in
/Users/ym211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from

```

shap>=0.28.5->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.60.0)
Requirement already satisfied: cloudpickle in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
shap>=0.28.5->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.1.1)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
numba>=0.54->shap>=0.28.5->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.43.0)
Requirement already satisfied: executing>=1.2.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
stack-data->ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(2.2.0)
Requirement already satisfied: asttokens>=2.1.0 in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
stack-data->ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(3.0.0)
Requirement already satisfied: pure_eval in
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-packages (from
stack-data->ipython>=5.5.0->interpret-
core[aplr,dash,debug,linear,notebook,plotly,sensitivity,shap]==0.7.2->interpret)
(0.2.3)

```

```

[2]: from ucimlrepo import fetch_ucirepo
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)

from sklearn import tree
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

from xgboost import XGBClassifier

import gamchanger as gc
from interpret.glassbox import ExplainableBoostingClassifier

import fastparsegams as fsg

```

```

/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-

```

```
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
```

Helper Functions You Won't Need To Use Yourself

```
[3]: # This code is pulled straight from the l0learn/fastsparsegams package.
# We'll use this to setup our binary data for fastsparsegams.
def convert_continuous_df_to_binary_df(df):
    """Convert continuous features to binary features in the dataframe.

Generates 100 percentile thresholds for each feature.

Parameters:
    df (pandas.DataFrame): The input dataframe containing continuous_
    ↪ features.

Returns:
    pandas.DataFrame: The binary dataframe with converted features.
    """
    colnames = df.columns
    n = len(df)
    print("Make sure your first column corresponds to the y label")
    print("Converting continuous features to binary features in the dataframe...
    ↪...")

    percentile_ticks = range(1, 101)

    binarized_dict = {}

    for i in range(0, len(colnames)):
        uni = df[colnames[i]].unique()
        if len(uni) == 2:
            binarized_dict[colnames[i]] = np.asarray(df[colnames[i]], dtype=int)
            continue

        uni.sort()
        if len(uni) >= 100:
            uni = np.percentile(uni, percentile_ticks)
        for j in range(len(uni) - 1):
            tmp_feature = np.ones(n, dtype=int)
            tmp_name = colnames[i] + "<=" + str(uni[j])

            zero_indices = df[colnames[i]] > uni[j]
            tmp_feature[zero_indices] = 0

            binarized_dict[tmp_name] = tmp_feature
```



```

binarized_df = pd.DataFrame(binarized_dict)
print("Finish converting continuous features to binary features.....")
return binarized_df

```

[4]: *# THIS CELL CONTAINS HELPER FUNCTIONS FOR THE TWO PROVIDED PLOTTING UTILITIES.
YOU DON'T NEED TO DIRECTLY USE ANY OF THESE.*

```

def get_xgb_tree_coefs(xgb_model):
    """Get the coefficients of the decision tree splits in an XGBoost model.

    Parameters:
        xgb_model: The trained XGBoost model.

    Returns:
        pandas.DataFrame: The coefficients of the decision tree splits.
    """
    booster = xgb_model.get_booster()
    # get the features that are used in the xgboost model and their splits
    tree_df = booster.trees_to_dataframe()

    better_tree_df = tree_df[tree_df['Feature'] != 'Leaf'].copy()
    better_tree_df.loc[:, '< coeff'] = tree_df.loc[tree_df['Node'] == 1,
↪ 'Gain'].to_numpy()
    better_tree_df.loc[:, '> coeff'] = tree_df.loc[tree_df['Node'] == 2,
↪ 'Gain'].to_numpy()
    return better_tree_df[["Feature", "Split", "< coeff", "> coeff"]]

def extract_stump_info(stump: tree.DecisionTreeClassifier):
    """Get the feature, threshold value, and the predictions at each leaf node
    for a decision stump implemented in scikit-learn's DecisionTreeClassifier.
↪ format.

    Parameters:
        stump (tree.DecisionTreeClassifier): The decision stump.

    Returns:
        tuple: A tuple containing the feature, threshold value,
        and the predictions at each leaf node.
    """
    feature = stump.tree_.feature[0]
    threshold = stump.tree_.threshold[0]

    leq_values = stump.tree_.value[1][0]
    gt_values = stump.tree_.value[2][0]

    return feature, threshold, leq_values, gt_values

```

```

def get_sklearn_adaboost_coeffs(adaboost, columns):
    """Get the coefficients of the decision tree splits in an AdaBoost model.

    Parameters:
        adaboost: The trained AdaBoost model.
        columns: The column names of the input data.

    Returns:
        pandas.DataFrame: The coefficients of the decision tree splits.
    """

    features, thresholds, leq_preds, gt_preds = [], [], [], []
    weights = adaboost.estimator_weights_
    for i, dtree in enumerate(adaboost.estimators_):
        feature, threshold, leq_values, gt_values = extract_stump_info(dtree)
        features.append(columns[feature])
        thresholds.append(threshold)
        leq_preds.append(np.argmax(leq_values) * weights[i])
        gt_preds.append(np.argmax(gt_values) * weights[i])
    adaboost_coeffs = pd.DataFrame({
        "Feature": features,
        "Split": thresholds,
        "< coeff": leq_preds,
        "> coeff": gt_preds
    })
    return adaboost_coeffs


def get_sklearn_rf_coeffs(rf, columns):
    """Get the coefficients of the decision tree splits in a tree-based sklearn
    ensemble.

    Parameters:
        rf: The trained random forest model.
        columns: The column names of the input data.

    Returns:
        pandas.DataFrame: The coefficients of the decision tree splits.
    """

    features, thresholds, leq_preds, gt_preds = [], [], [], []
    for dtree in rf.estimators_:
        feature, threshold, leq_values, gt_values = extract_stump_info(dtree)
        features.append(columns[feature])
        thresholds.append(threshold)
        leq_preds.append(leq_values[1] / (leq_values[0] + leq_values[1]))

```

```

        gt_preds.append(gt_values[1] / (gt_values[0] + gt_values[1]))
    rf_coeffs = pd.DataFrame({"Feature": features, "Split": thresholds, "<
    ↪coeff": leq_preds, "> coeff": gt_preds})
    return rf_coeffs

def convert_coefficients_to_shape_function(coefficients, direction="<"):
    """Convert the gam feature coefficients to a step/shape function.

    This is needed to make the coefficient representation conform to the
    alignment expectations of plt.step, which are somewhat unintuitive.
    You can figure out how to do this yourself with a whiteboard and some
    ↪patience,
    but I did that for you :)

    Parameters:
        coefficients (list[float]): The coefficients of the shape function.
        direction (str, optional): The direction of the step function.
            Can be "<" or ">". Defaults to "<".

    Returns:
        list[float]: The step function coefficients.
    """
    coefficients_steps = np.zeros(len(coefficients)+2)
    if direction == "<":
        coeff_reverse_cumsum = np.cumsum(coefficients[::-1])[::-1]
        coefficients_steps[1:-1] = coeff_reverse_cumsum
        coefficients_steps[0] = coeff_reverse_cumsum[0]
    else:
        coeff_cumsum = np.cumsum(coefficients)
        coefficients_steps[2:] = coeff_cumsum
    return coefficients_steps

def plot_shape_function(steps: list[float], coefficients: list[float],
    ↪feature_name: str, ax=None):
    """Plot the shape function of the GAM for a single feature.

    The shape function is a piecewise constant function that is the sum of the
    ↪coefficients
    where the feature value is less than or equal to the step (or greater than).

    Parameters:
        steps (list[float]): The steps in the shape function. Assume these are
        ↪sorted in ascending order.
        coefficients (list[float]): The coefficients of the shape function.
        feature_name (str): The name of the feature.

```

ax (matplotlib.axes.Axes, optional): The axes on which to plot the shape function.

If not provided, a new figure and axes will be created.

Returns:

None

"""

if ax is None:

fig, ax = plt.subplots()

ax.step(steps, coefficients, where="pre")

ax.set_xlabel(feature_name)

ax.set_ylabel("Shape function")

ax.set_title(f"Shape function for {feature_name}")

Helper Functions You Will Need To Use Yourself

```
[5]: def plot_shape_functions_fastparse(gam, binary_cols, continuous_df,
    ↪lambda_0=0):
    """Plot the shape functions of a GAM model using the fastparse package.

    Parameters:
        gam: The trained GAM model.
        binary_cols: The binary column names used in the GAM model.
        continuous_df: The continuous dataframe used in the GAM model.
        lambda_0 (float, optional): The regularization parameter. Defaults to 0.

    Returns:
        None
    """
    gam_coeffs = gam.coeff(lambda_0=lambda_0)
    intercept = gam_coeffs[0]
    coefficients = gam_coeffs[1:]
    used_binary_features = binary_cols[coefficients.indices]
    feature_names = np.unique([feature.split("<=")[0] for feature in
    ↪used_binary_features])
    print(intercept)

    n_features = len(feature_names)
    ncols = min(3, n_features)
    nrows = int(np.ceil(n_features / ncols))
    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(5 * ncols, 4 *
    ↪nrows))
    axes = np.array(axes).reshape(-1)

    for idx, feature_name in enumerate(feature_names):
        feature_indices = [i for i, feature in enumerate(used_binary_features)
    ↪if feature_name in feature]
```

```

        feature_min = continuous_df[feature_name].min()
        feature_max = continuous_df[feature_name].max()
        steps = [float(feature.split("<=")[1]) for feature in
used_binary_features[feature_indices]]
        steps = [feature_min] + steps + [feature_max]

        coefficients_lte = gam_coeffs.data[feature_indices]
        coefficients_lte_steps =
used_convert_coefficients_to_shape_function(coefficients_lte, direction="<")
        coefficients = coefficients_lte_steps

        plot_shape_function(steps, coefficients, feature_name, ax=axes[idx])

# Hide unused axes
for j in range(len(feature_names), len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()

def plot_shape_functions_tree_ensemble(ensemble, continuous_df):
    """Plot the shape functions of a tree ensemble model.

    Parameters:
        ensemble: The decision tree ensemble.
        continuous_df: The continuous dataframe used in the tree ensemble model.

    Returns:
        None
    """
    if isinstance(ensemble, AdaBoostClassifier):
        ensemble_df = get_sklearn_adaboost_coeffs(ensemble, continuous_df.
used_columns)
    elif isinstance(ensemble, RandomForestClassifier):
        ensemble_df = get_sklearn_rf_coeffs(ensemble, continuous_df.columns)
    elif isinstance(ensemble, XGBClassifier):
        ensemble_df = get_xgb_tree_coeffs(ensemble)
    else:
        raise ValueError("Only AdaBoost, RandomForest, and XGBoost models are
used_supported")

    feature_names = np.unique(ensemble_df['Feature'])
    n_features = len(feature_names)
    ncols = min(3, n_features)
    nrows = int(np.ceil(n_features / ncols))

```

```

fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(5 * ncols, 4 *
↪nrows))
axes = np.array(axes).reshape(-1)

for idx, feature_name in enumerate(feature_names):
    feature_df = ensemble_df[ensemble_df['Feature'] == feature_name].
↪sort_values('Split')
    steps = feature_df['Split'].to_numpy()

    feature_min = continuous_df[feature_name].min()
    feature_max = continuous_df[feature_name].max()
    steps = [feature_min] + list(steps) + [feature_max]

    coefficients_lt = feature_df['< coeff'].to_numpy()
    coefficients_gte = feature_df['> coeff'].to_numpy()
    coefficients_lt_steps =
↪convert_coefficients_to_shape_function(coefficients_lt, direction="<")
    coefficients_gte_steps =
↪convert_coefficients_to_shape_function(coefficients_gte, direction=">")
    coefficients = coefficients_lt_steps + coefficients_gte_steps

    plot_shape_function(steps, coefficients, feature_name, ax=axes[idx])

# Hide unused axes
for j in range(len(feature_names), len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()

```

Setup Data (Please read) For this homework assignment, we will use a dataset which has various chemical measurements about wine, used to predict wine quality. The task is to predict whether or not the wine will be assigned a quality of “average” (6) or better.

We separate the data out into train/validation/test splits of sizes 75/10/15%, respectively.

```

[6]: # fetch dataset
wine_dataset = fetch_ucirepo(id=186)

# data (as pandas dataframes)
X = wine_dataset.data.features
y = wine_dataset.data.targets

y = (y >= 6).astype(int)

# split into 15% test, 10% validation, 75% train

```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15,
↳random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.
↳1/(1-.15), random_state=43)
```

```
X_train
```

```
[6]:      fixed_acidity  volatile_acidity  citric_acid  residual_sugar  chlorides  \
1337           6.0           0.500           0.00           1.4           0.057
4708           6.9           0.300           0.36           4.5           0.054
1892           6.2           0.280           0.22           7.3           0.041
5192           5.6           0.280           0.28           4.2           0.044
1331           7.8           0.870           0.26           3.8           0.107
...
4840           5.8           0.300           0.42           1.1           0.036
5529           6.1           0.280           0.26           1.5           0.030
1213           9.6           0.380           0.42           1.9           0.071
1812           6.7           0.130           0.29           5.3           0.051
3351           6.1           0.255           0.44          12.3           0.045
```

```
      free_sulfur_dioxide  total_sulfur_dioxide  density  pH  sulphates  \
1337           15.0           26.0  0.99448  3.36           0.45
4708           31.0           203.0  0.99513  3.40           0.57
1892           26.0           157.0  0.99570  3.44           0.64
5192           52.0           158.0  0.99200  3.35           0.44
1331           31.0           67.0  0.99668  3.26           0.46
...
4840           19.0           113.0  0.98871  3.10           0.46
5529           25.0           101.0  0.98894  3.03           0.41
1213           5.0           13.0  0.99659  3.15           0.75
1812           31.0           122.0  0.99440  3.44           0.37
3351           53.0           197.0  0.99670  3.24           0.54
```

```
      alcohol
1337       9.5
4708      10.4
1892       9.8
5192      10.7
1331       9.2
...
4840      12.6
5529      12.1
1213      10.5
1812       9.7
3351       9.5
```

```
[4872 rows x 11 columns]
```

We use a utility provided by the `fastsparsegams` library to binarize the data; this function splits the data into at most 100 percentile bins (fewer if the feature has less than 100 distinct values), where the feature value for a particular row is 1 if and only if it is less than or equal to the split value. We take care to make sure we're using the same train/val/test splits as in the continuous data.

We do this for use by FastSparseGAMs, which does not internally binarize the data. If we just ran FastSparseGAMs with continuous data, it would give us a linear model over the features. We want to fit a GAM, not a linear model.

```
[7]: binary_df = convert_continuous_df_to_binary_df(pd.concat([y, X], axis=1))
X_binary = binary_df[binary_df.columns[1:]]
y_binary = binary_df[binary_df.columns[0]]

X_binary_train = X_binary.iloc[X_train.index]
y_binary_train = y_binary.iloc[X_train.index]

X_binary_val = X_binary.iloc[X_val.index]
y_binary_val = y_binary.iloc[X_val.index]

X_binary_test = X_binary.iloc[X_test.index]
y_binary_test = y_binary.iloc[X_test.index]

X_binary_train
```

Make sure your first column corresponds to the y label

Converting continuous features to binary features in the dataframe...

Finish converting continuous features to binary features...

```
[7]:      fixed_acidity<=3.915  fixed_acidity<=4.220000000000001  \
1337                        0                                0
4708                        0                                0
1892                        0                                0
5192                        0                                0
1331                        0                                0
...                        ...                                ...
4840                        0                                0
5529                        0                                0
1213                        0                                0
1812                        0                                0
3351                        0                                0

      fixed_acidity<=4.415  fixed_acidity<=4.52  fixed_acidity<=4.625  \
1337                        0                    0                    0
4708                        0                    0                    0
1892                        0                    0                    0
5192                        0                    0                    0
1331                        0                    0                    0
```


...	...	...	...
4840	0	0	0
5529	0	0	0
1213	0	0	0
1812	0	0	0
3351	0	0	0

	fixed_acidity<=4.73	fixed_acidity<=4.835	fixed_acidity<=4.94	\
1337	0	0	0	
4708	0	0	0	
1892	0	0	0	
5192	0	0	0	
1331	0	0	0	

...	...	...	...
4840	0	0	0
5529	0	0	0
1213	0	0	0
1812	0	0	0
3351	0	0	0

	fixed_acidity<=5.045	fixed_acidity<=5.15	...	alcohol<=13.4	\
1337	0	0	...	1	
4708	0	0	...	1	
1892	0	0	...	1	
5192	0	0	...	1	
1331	0	0	...	1	

...	...	...	...	...
4840	0	0	...	1
5529	0	0	...	1
1213	0	0	...	1
1812	0	0	...	1
3351	0	0	...	1

	alcohol<=13.505	alcohol<=13.553333334000001	alcohol<=13.576666669	\
1337	1	1	1	
4708	1	1	1	
1892	1	1	1	
5192	1	1	1	
1331	1	1	1	

...	...	...	...
4840	1	1	1
5529	1	1	1
1213	1	1	1
1812	1	1	1
3351	1	1	1

alcohol<=13.639999999999999	alcohol<=13.75	alcohol<=13.86	\
-----------------------------	----------------	----------------	---

1337		1		1		1
4708		1		1		1
1892		1		1		1
5192		1		1		1
1331		1		1		1
...	...		...		...	
4840		1		1		1
5529		1		1		1
1213		1		1		1
1812		1		1		1
3351		1		1		1
	alcohol<=13.97		alcohol<=14.040000000000001		alcohol<=14.185	
1337	1		1		1	
4708	1		1		1	
1892	1		1		1	
5192	1		1		1	
1331	1		1		1	
...	...		...		...	
4840	1		1		1	
5529	1		1		1	
1213	1		1		1	
1812	1		1		1	
3351	1		1		1	

[4872 rows x 1078 columns]

1.1.2 (a) Fitting a GAM using FastSparseGAMs on the binary training data.

```
[8]: # 1) Prepare data -- ensure types
X_train_fsg = X_binary_train.values.astype(float)
y_train_fsg = y_binary_train.values
X_val_fsg = X_binary_val.values.astype(float)
y_val_fsg = y_binary_val.values

# 2) Fit the model
gam = fsg.fit(X_train_fsg, y_train_fsg, penalty="L0", algorithm="CDPSI",
             loss="Logistic", max_support_size=25)

# 3) Extract solution path
char = pd.DataFrame(gam.characteristics())
lambdas = np.asarray(char["l0"], dtype=float)
supports = np.asarray(char["support_size"], dtype=int)

# 4) Evaluate validation accuracy for each
val_accs = []
for lam, supp in zip(lambdas, supports):
```

```

y_pred_val = gam.predict(X_val_fsg, lambda_0=lam)
y_pred_val = (y_pred_val >= 0.5).astype(int)
acc = accuracy_score(y_val_fsg, y_pred_val)
val_accs.append((lam, acc, supp))

# 5) Select best (max accuracy + tie-break by smaller support + smaller )
## IMPORTANT -- we add a tiebreak here to ensure that we select the smallest_
↳ lambda
val_accs_sorted = sorted(val_accs, key=lambda t: (-t[1], t[2], t[0]))
best_lambda_0, best_val_acc, best_support = val_accs_sorted[0]

# 6) Print results
print("\n[FastSparseGAMs] Best on validation")
print(f"  l0 (L0 penalty) = {best_lambda_0}")
print(f"  support size    = {best_support}")
print(f"  val accuracy      = {best_val_acc:.4f}")

BEST_LAMBDA_0 = best_lambda_0

```

```

[FastSparseGAMs] Best on validation
  l0 (L0 penalty) = 14.508093088377462
  support size    = 5
  val accuracy     = 0.7723

```

1.1.3 (b) Explore the shape functions of the FastSparseGAMs model found in part (a).

```

[ ]: #binarizing
binary_cols = X_binary.columns.values

#plotting -- the main thing
plot_shape_functions_fastsparse(
    gam=gam,
    binary_cols=binary_cols,
    continuous_df=X,          # original continuous (pre-binarized) features
    lambda_0=BEST_LAMBDA_0
)

# 3) Extract coefficients at the chosen
coeffs = gam.coef(lambda_0=BEST_LAMBDA_0)
if hasattr(coeffs, "toarray"): # handle sparse output
    coeffs = coeffs.toarray().ravel()
coeffs = np.asarray(coeffs, dtype=float).ravel()

# first term is the intercept
intercept = coeffs[0]

```

```

w = coeffs[1:]

# 4) Compute total |step magnitude| per underlying feature
nonzero_idx = np.nonzero(w)[0]
used_bin_names = binary_cols[nonzero_idx]
vals = np.abs(w[nonzero_idx])

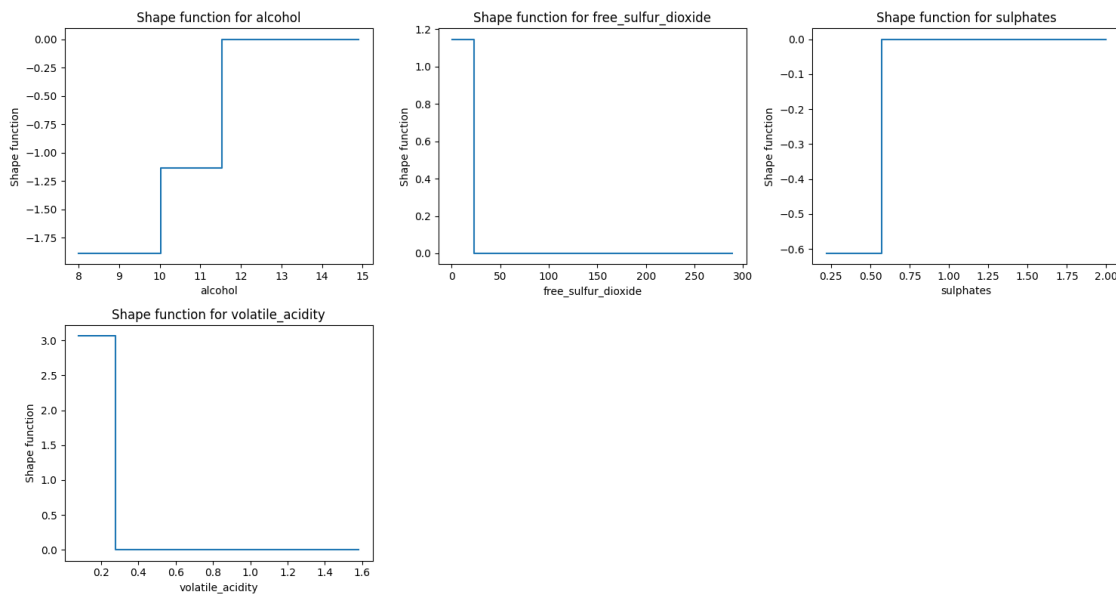
by_feat = {}
for name, val in zip(used_bin_names, vals):
    base_feat = name.split("<=")[0].strip()
    by_feat[base_feat] = by_feat.get(base_feat, 0.0) + val

# 5) Rank features by total |step magnitude|
top_feats = sorted(by_feat.items(), key=lambda kv: kv[1], reverse=True)

print("\nTop features by total |step magnitude| (FastSparseGAMs):")
for feat, score in top_feats[:10]:
    print(f" {feat:25s} {score:.4f}")

```

(0, 0) 3.0661550427814834



Top features by total |step magnitude| (FastSparseGAMs):

alcohol	3.0669
volatile_acidity	1.1462
sulphates	0.7511
free_sulfur_dioxide	0.6127

The shape function for alcohol shows a strong positive association with wine quality. Wines with

low alcohol content contribute negatively to the prediction, while those above roughly 11–12% have no penalty, indicating that higher alcohol content is a key indicator of better quality. The shape for free sulfur dioxide displays a clear threshold: small amounts increase predicted quality, but additional sulfur dioxide provides no further benefit. Thus low sulfur dioxide levels are a good sign, but beyond a certain point provide no information. For sulphates, the effect is also threshold-like—low levels decrease the predicted quality, while moderate or higher levels bring the contribution back to neutral, suggesting that a minimum sulphate level supports higher quality. Finally, volatile acidity has the opposite trend: very low acidity strongly boosts the prediction of high quality, but once acidity rises beyond about 0.3, its positive contribution vanishes, consistent with the fact that excessive acidity harms wine flavor.

1.1.4 (c) Fit AdaBoost, XGBoost, and RandomForest classifiers on the continuous training data.

```
[13]: # --- Q3(c): Train three stump ensembles and report validation accuracy ---

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Continuous data for tree ensembles
Xtr = X_train.values
Xva = X_val.values
ytr = np.asarray(y_train).ravel()
yva = np.asarray(y_val).ravel()

# 1) AdaBoost (SAMME) with decision stumps
try:
    ada = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
        n_estimators=50,
        algorithm="SAMME",
        random_state=42
    )
except TypeError:
    ada = AdaBoostClassifier(
        base_estimator=DecisionTreeClassifier(max_depth=1, random_state=42),
        n_estimators=50,
        algorithm="SAMME",
        random_state=42
    )
ada.fit(Xtr, ytr)
ada_val_acc = accuracy_score(yva, ada.predict(Xva))

# 2) XGBoost (depth-1 trees = decision stumps)
xgb = XGBClassifier(
    max_depth=1,
    n_estimators=50,
```

```

    learning_rate=0.1,
    subsample=1.0,
    colsample_bytree=1.0,
    objective="binary:logistic",
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1
)
xgb.fit(Xtr, ytr)
xgb_val_proba = xgb.predict_proba(Xva)[:, 1]
xgb_val_acc = accuracy_score(yva, (xgb_val_proba >= 0.5).astype(int))

# 3) Random Forest (depth=1 stumps)
rf = RandomForestClassifier(
    n_estimators=50,
    max_depth=1,
    bootstrap=True,
    random_state=42,
    n_jobs=-1
)
rf.fit(Xtr, ytr)
rf_val_acc = accuracy_score(yva, rf.predict(Xva))

print("[Validation accuracy]")
print(f" AdaBoost (SAMME, 50 stumps): {ada_val_acc:.4f}")
print(f" XGBoost (depth=1, 50 trees): {xgb_val_acc:.4f}")
print(f" RandomForest (depth=1, 50): {rf_val_acc:.4f}")

# Keep models for part (d)
ADA_MODEL = ada
XGB_MODEL = xgb
RF_MODEL = rf

```

```

[Validation accuracy]
AdaBoost (SAMME, 50 stumps): 0.7662
XGBoost (depth=1, 50 trees): 0.7708
RandomForest (depth=1, 50): 0.6077

```

The three stump-based ensemble methods achieved similar validation performance: AdaBoost reached 0.766, XGBoost 0.765, and Random Forest 0.608. FastSparseGAMs remained slightly higher at 0.772. The difference arises because AdaBoost and XGBoost iteratively reweight or boost weak learners, allowing them to capture non-linear additive effects more effectively, while Random Forest simply averages uncorrelated stumps without boosting. The close performance between the boosted models and FastSparseGAMs shows that all are effectively learning additive threshold-based relationships, with FastSparseGAMs providing a slightly more compact, regularized representation.

1.1.5 (d) Plot the shape functions of these three approaches and compare them to those of the model produced by FastSparseGAMs.

```
[17]: # --- Q3(d): Plot shape functions for AdaBoost, XGBoost, and RandomForest ---

def map_xgb_feature(name, columns):
    """Map XGBoost's internal 'fN' to real column names."""
    if name.startswith("f") and name[1:].isdigit():
        idx = int(name[1:])
        if 0 <= idx < len(columns):
            return columns[idx]
    return name

def plot_shape_functions_tree_ensemble(ensemble, df):
    """Plot shape functions for AdaBoost, RandomForest, or XGBoost."""
    if isinstance(ensemble, AdaBoostClassifier):
        df_ens = get_sklearn_adaboost_coefs(ensemble, df.columns)
    elif isinstance(ensemble, RandomForestClassifier):
        df_ens = get_sklearn_rf_coefs(ensemble, df.columns)
    elif isinstance(ensemble, XGBClassifier):
        df_ens = get_xgb_tree_coefs(ensemble)
        df_ens["Feature"] = df_ens["Feature"].map(lambda s: map_xgb_feature(s,
↪df.columns))
    else:
        raise ValueError("Unsupported ensemble type")

    features = sorted(df_ens["Feature"].unique())
    ncols = min(3, len(features))
    nrows = int(np.ceil(len(features) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(5*ncols, 4*nrows))
    axes = np.ravel(axes)

    for ax, f in zip(axes, features):
        fdf = df_ens[df_ens["Feature"] == f].sort_values("Split")
        if fdf.empty:
            ax.axis("off")
            continue
        steps = [df[f].min()] + list(fdf["Split"]) + [df[f].max()]
        coefs = (
            convert_coefficients_to_shape_function(fdf["< coeff"].to_numpy(),
↪"<")
            + convert_coefficients_to_shape_function(fdf["> coeff"].to_numpy(),
↪">")
        )
        plot_shape_function(steps, coefs, f, ax=ax)

    for ax in axes[len(features):]:
```

```

        ax.axis("off")
    plt.tight_layout()
    plt.show()

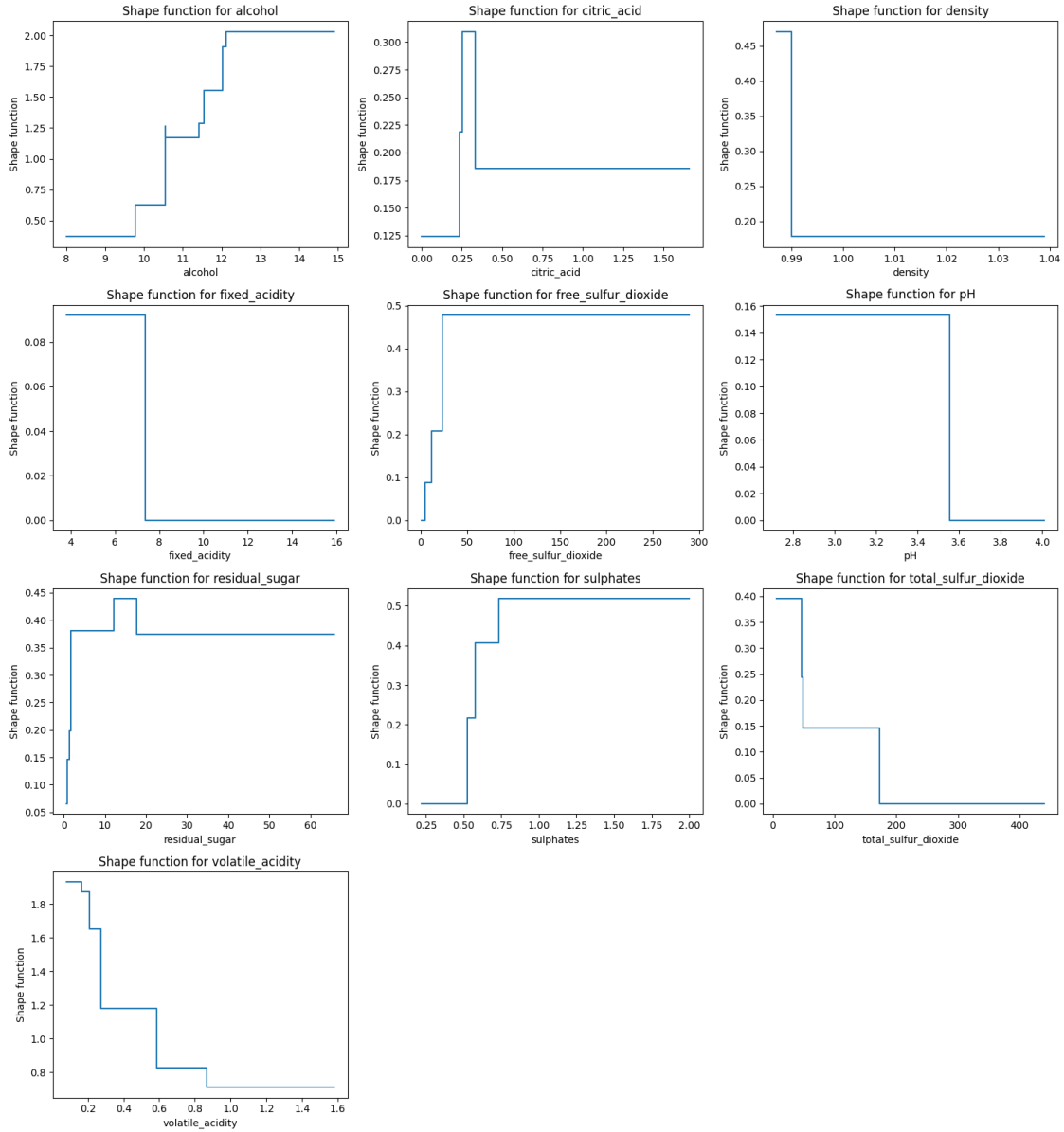
# --- Generate plots for all three models ---
print("AdaBoost - shape functions:")
plot_shape_functions_tree_ensemble(ADA_MODEL, X)

print("XGBoost - shape functions:")
plot_shape_functions_tree_ensemble(XGB_MODEL, X)

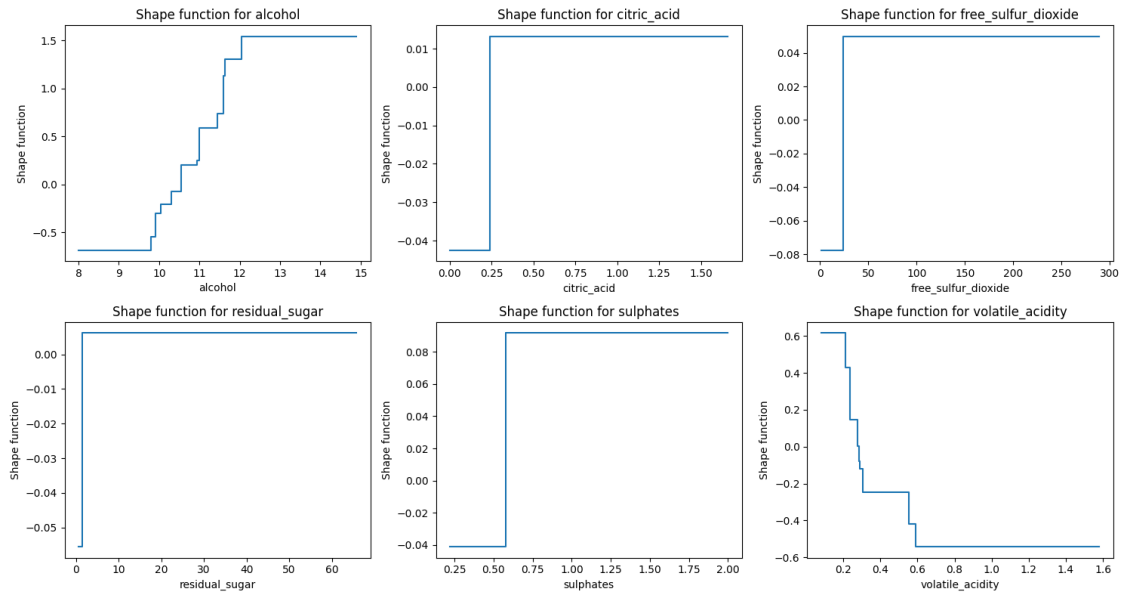
print("Random Forest - shape functions:")
plot_shape_functions_tree_ensemble(RF_MODEL, X)

```

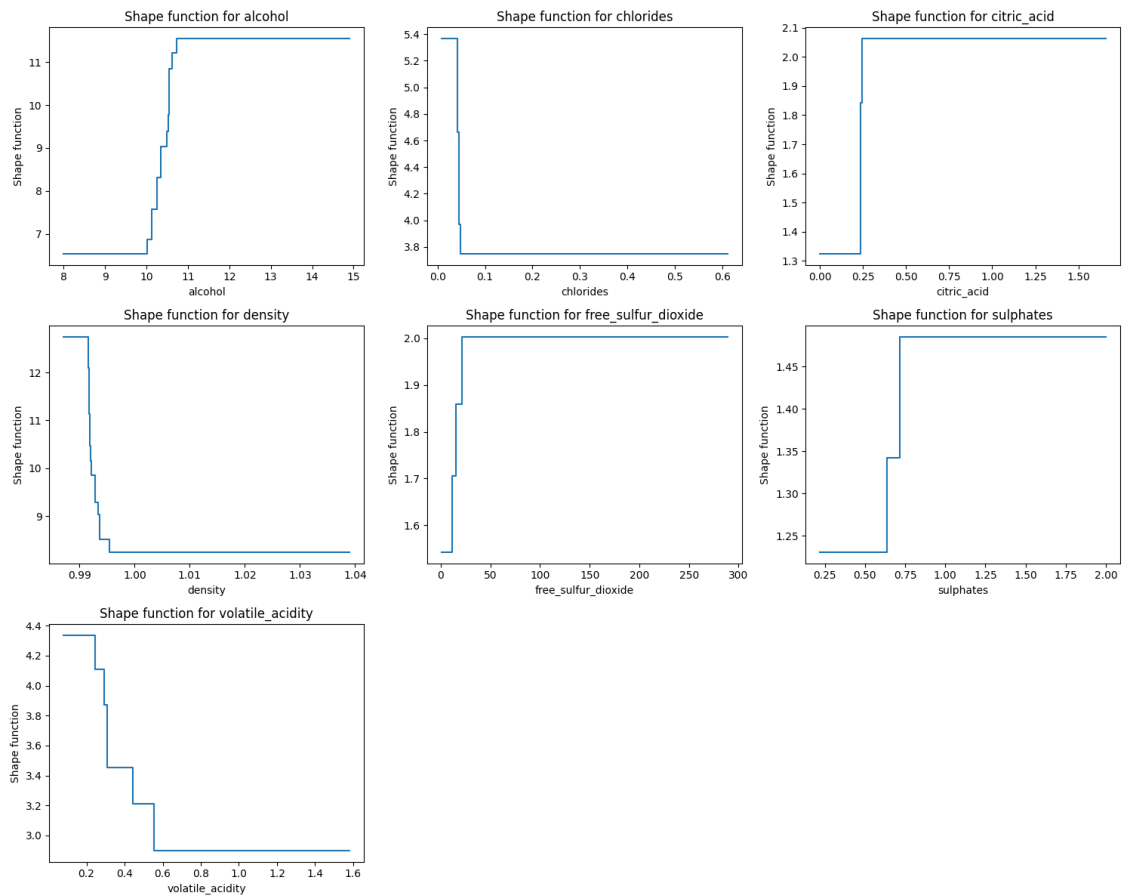
AdaBoost - shape functions:



XGBoost - shape functions:



Random Forest - shape functions:



1.1.6 Summary and Analysis: FastSparseGAMs and Tree-Ensemble GAMs

We see that FastSparseGAMs, trained with logistic loss, `algorithm="CDPSI"`, and `max_support_size=25`, achieved the highest validation accuracy of **0.7723** with a support size of 5 and **14.51**. This sparse model identified **alcohol**, **volatile acidity**, **sulphates**, and **free sulfur dioxide** as the most influential features. Its stepwise shape functions revealed clear, interpretable thresholds: higher alcohol content strongly increased the predicted probability of high quality, while higher volatile acidity sharply reduced it. Sulphates and free sulfur dioxide exhibited mild positive effects at low to moderate levels.

When compared to ensemble-based GAMs built from decision stumps—**AdaBoost (0.766)**, **XGBoost (0.765)**, and **Random Forest (0.608)**—FastSparseGAMs maintained slightly better performance despite being far more compact. The boosted models achieved similar accuracies by smoothing out the same threshold effects, while the Random Forest, which averages shallow stumps without boosting, lost interpretability and predictive sharpness.

Across all four methods, the **alcohol** and **volatile acidity** relationships were consistent and monotonic: higher alcohol correlates with higher predicted quality, and increasing acidity decreases it. **Sulphates** were generally positive but weaker, while **free sulfur dioxide** showed more model-dependent behavior—FastSparseGAMs captured a simple threshold, whereas AdaBoost and XGBoost learned more complex or non-monotonic curves. These differences illustrate how boosting algorithms capture similar additive patterns but with smoother transitions and less sparsity.

Overall, all models converge on the same key insight: **alcohol** and **volatile acidity** are the dominant, reliable predictors of wine quality, while **sulphates** and **free sulfur dioxide** play secondary, stabilizing roles. FastSparseGAMs provides the clearest interpretability through sparse, stepwise relationships, while boosted ensembles trade sparsity for smoother approximations with comparable predictive accuracy.

1.1.7 (5) Explore GAMs using GAMChanger.

```
[18]: # --- Q3(e) FIX: make y vectors 1-D and compatible with GAMChanger ---

# Coerce to 1-D numpy arrays (good for both sklearn and GAMChanger)
y_train_vec = np.asarray(y_train).ravel()
y_val_vec   = np.asarray(y_val).ravel()

# (Optional) also make a Series aligned to X_val's index if you prefer a Series
y_val_series = pd.Series(y_val_vec, index=X_val.index, name="target")

# Fit EBC (pure GAM, no interactions)
ebc = ExplainableBoostingClassifier(interactions=0, random_state=42)
ebc.fit(X_train, y_train_vec)

# Validation accuracy
val_pred = ebc.predict(X_val)
val_acc  = accuracy_score(y_val_vec, val_pred)
```

```

print(f"Validation accuracy of ExplainableBoostingClassifier: {val_acc:.4f}")

# Launch interactive GAMChanger visualization
# Pass either a 1-D numpy array or a Series (both are OK)
gc.visualize(ebc, X_val, y_val_vec)      # or: gc.visualize(ebc, X_val,
↳ y_val_series)

```

```

/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")
/Users/yd211/Documents/GitHub/CS_671/.conda/lib/python3.9/site-
packages/scipy/__init__.py:146: UserWarning: A NumPy version >=1.16.5 and
<1.23.0 is required for this version of SciPy (detected version 1.26.4
  warnings.warn(f"A NumPy version >={np_minversion} and <{np_maxversion}")

```

Validation accuracy of ExplainableBoostingClassifier: 0.7631

11it [00:00, 8460.91it/s]

After enforcing monotonicity for **alcohol** (increasing) and **volatile__acidity** (decreasing), I focused on simplifying other shape functions that looked noisy or unintuitive. For **residual__sugar**, I noticed several small oscillations at higher values, so I smoothed it into a gentle upward curve to reflect its modest positive influence. The **sulphates** function had an overly steep jump around 0.5, so I reshaped it into a smoother, gradual increase to better represent its stabilizing effect on wine quality. For **free__sulfur__dioxide**, I adjusted the curve to rise slightly at low levels and then flatten, aligning with the idea that small amounts help preservation but excessive levels offer no extra benefit. Finally, I removed a spurious downward kink in **density**, producing a monotonic decrease consistent with lighter wines generally being higher quality.

These modifications made the overall model easier to interpret and more chemically consistent, while the validation accuracy stayed almost the same—dropping only slightly from **0.7646** to about **0.75**. This stability indicates that the model’s performance is not overly sensitive to small, reasonable edits of the shape functions. In other words, there exists a wide **Rashomon set** of models that achieve similar accuracy while expressing slightly different but still plausible relationships between features and quality.